

-- MySQL Workbench Forward Engineering

```
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
```

-- Schema mydb

-- Schema mydb

```
CREATE SCHEMA IF NOT EXISTS `hefesto` DEFAULT CHARACTER SET utf8mb3 ;
SHOW WARNINGS;
USE `hefesto` ;
```

-- Table `cliente`

```
CREATE TABLE IF NOT EXISTS `cliente` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome_cliente` VARCHAR(50) NOT NULL,
  `cnpj_cliente` VARCHAR(14) NOT NULL,
  `email_cliente` VARCHAR(50) NOT NULL,
  `tel_cliente` VARCHAR(11) NOT NULL,
  `cep_cliente` VARCHAR(8) NOT NULL,
  `ativado_cliente` TINYINT NOT NULL DEFAULT '0',
  PRIMARY KEY (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 8
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

-- Table `fornecedor`

```
CREATE TABLE IF NOT EXISTS `fornecedor` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome_fornecedor` VARCHAR(100) NOT NULL,
  `cnpj_fornecedor` VARCHAR(14) NOT NULL,
  `email_fornecedor` VARCHAR(50) NOT NULL,
  `tel_fornecedor` VARCHAR(11) NOT NULL,
  `cep_fornecedor` VARCHAR(8) NOT NULL,
```

```
`materia_prima_fornecedor` VARCHAR(50) NOT NULL,  
`ativado_fornecedor` TINYINT NOT NULL DEFAULT '0',  
PRIMARY KEY (`id`))
```

```
ENGINE = InnoDB
```

```
AUTO_INCREMENT = 15
```

```
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
-- -----  
-- Table `entrega_fornecedor`  
-- -----
```

```
CREATE TABLE IF NOT EXISTS `entrega_fornecedor` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `valor_total_entrega_fornecedor` DECIMAL(10,2) NOT NULL,  
  `valor_unt_entrega_fornecedor` DECIMAL(10,2) NOT NULL,  
  `item_entrega_fornecedor` VARCHAR(50) NOT NULL,  
  `quantidade_entrega_fornecedor` INT NOT NULL,  
  `observacao_entrega_fornecedor` VARCHAR(100) NULL DEFAULT NULL,  
  `fornecedor_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `fk_entrega_fornecedor_fornecedor1`  
    FOREIGN KEY (`fornecedor_id`)  
    REFERENCES `fornecedor` (`id`))
```

```
ENGINE = InnoDB
```

```
AUTO_INCREMENT = 8
```

```
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
-- -----  
-- Table `funcionario`  
-- -----
```

```
CREATE TABLE IF NOT EXISTS `funcionario` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `nome_funcionario` VARCHAR(100) NOT NULL,  
  `tel_funcionario` VARCHAR(11) NOT NULL,  
  `email_funcionario` VARCHAR(50) NOT NULL,  
  `cpf_funcionario` VARCHAR(11) NOT NULL,  
  `classificacao_funcionario` VARCHAR(50) NOT NULL,  
  `cep_funcionario` VARCHAR(8) NOT NULL,  
  `foto_funcionario` LONGTEXT NULL DEFAULT NULL,  
  `num_clt_funcionario` VARCHAR(11) NOT NULL,  
  `senha_funcionario` VARCHAR(200) NOT NULL,  
  `ativado_funcionario` TINYINT NOT NULL DEFAULT '0',  
  PRIMARY KEY (`id`))
```

```
ENGINE = InnoDB
```

```
AUTO_INCREMENT = 8
```

```
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
-- -----  
-- Table `pedido_cliente`  
-- -----
```

```
CREATE TABLE IF NOT EXISTS `pedido_cliente` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `d_realizacao_pedido_cliente` DATE NOT NULL,  
  `quantidade_pedido_cliente` INT NOT NULL,  
  `valor_unt_pedido_cliente` DECIMAL(10,2) NOT NULL,  
  `valor_total_pedido_cliente` DECIMAL(10,2) NOT NULL,  
  `prazo_final_pedido_cliente` DATE NOT NULL,  
  `op_pedido_cliente` TINYINT NOT NULL DEFAULT '0',  
  `funcionario_id` INT NOT NULL,  
  `cliente_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `fk_pedido_cliente_cliente1`  
    FOREIGN KEY (`cliente_id`)  
    REFERENCES `cliente` (`id`),  
  CONSTRAINT `fk_pedido_cliente_funcionario1`  
    FOREIGN KEY (`funcionario_id`)  
    REFERENCES `funcionario` (`id`))  
ENGINE = InnoDB  
AUTO_INCREMENT = 15  
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
-- -----  
-- Table `producao`  
-- -----
```

```
CREATE TABLE IF NOT EXISTS `producao` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `etapa_producao` VARCHAR(50) NOT NULL,  
  `status_producao` TINYINT NOT NULL DEFAULT '0',  
  `acontecimento_producao` DATETIME NOT NULL,  
  `funcionario_id` INT NOT NULL,  
  `cliente_id` INT NOT NULL,  
  `pedido_cliente_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `fk_producao_cliente1`  
    FOREIGN KEY (`cliente_id`)  
    REFERENCES `cliente` (`id`),  
  CONSTRAINT `fk_producao_funcionario1`  
    FOREIGN KEY (`funcionario_id`)  
    REFERENCES `funcionario` (`id`),
```

```
CONSTRAINT `fk_producao_pedido_cliente1`  
  FOREIGN KEY (`pedido_cliente_id`)  
    REFERENCES `pedido_cliente` (`id`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION)  
ENGINE = InnoDB  
AUTO_INCREMENT = 8  
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
--  
-----  
-- Table `estoque_final`  
-----
```

```
CREATE TABLE IF NOT EXISTS `estoque_final` (  
  `id` INT UNSIGNED NOT NULL,  
  `barracao_Rua_estoque_final` VARCHAR(2) NOT NULL,  
  `barracao_andar_estoque_final` VARCHAR(2) NOT NULL,  
  `barracao_num_estoque_final` VARCHAR(2) NOT NULL,  
  `producao_id` INT NOT NULL,  
  PRIMARY KEY (`id`),  
  CONSTRAINT `fk_estoque_final_producao1`  
    FOREIGN KEY (`producao_id`)  
      REFERENCES `producao` (`id`))  
ENGINE = InnoDB  
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
--  
-----  
-- Table `historico_compra`  
-----
```

```
CREATE TABLE IF NOT EXISTS `historico_compra` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `d_entrega_historico_compra` DATETIME NOT NULL,  
  `d_realizacao_historico_compra` DATE NOT NULL,  
  `entrega_fornecedor_id` INT NOT NULL,  
  PRIMARY KEY (`id`))  
ENGINE = InnoDB  
AUTO_INCREMENT = 50  
DEFAULT CHARACTER SET = utf8mb3;
```

```
SHOW WARNINGS;
```

```
--  
-----  
-- Table `historico_vendas`  
-----
```

```
CREATE TABLE IF NOT EXISTS `historico_vendas` (  
  `id` INT NOT NULL AUTO_INCREMENT,
```

```

`id` INT NOT NULL AUTO_INCREMENT,
`d_entrega_historico_vendas` DATE NOT NULL,
`d_realizacao_historico_vendas` DATE NOT NULL,
`quantidade_historico_vendas` INT NOT NULL,
`valor_total_historico_vendas` DECIMAL(10,2) NOT NULL,
`valor_unt_historico_vendas` DECIMAL(10,2) NOT NULL,
`prazo_final_historico_vendas` DATE NOT NULL,
`d_termino_historico_vendas` DATETIME NOT NULL,
`pedido_cliente_id` INT NOT NULL,
`estoque_final_id` INT UNSIGNED NOT NULL,
PRIMARY KEY (`id`),
CONSTRAINT `fk_historico_vendas_estoque_final1`
  FOREIGN KEY (`estoque_final_id`)
  REFERENCES `estoque_final` (`id`),
CONSTRAINT `fk_historico_vendas_pedido_cliente1`
  FOREIGN KEY (`pedido_cliente_id`)
  REFERENCES `pedido_cliente` (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 8
DEFAULT CHARACTER SET = utf8mb3;

```

SHOW WARNINGS;

```

-----
-- Table `produto`
-----

```

```

CREATE TABLE IF NOT EXISTS `produto` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome_produto` VARCHAR(50) NOT NULL,
  `dimensoes_produto` VARCHAR(100) NOT NULL,
  `imagem_produto` LONGTEXT NOT NULL,
  `minimo_produto` INT NOT NULL,
  `ativado_produto` TINYINT NOT NULL DEFAULT '0',
  PRIMARY KEY (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 15
DEFAULT CHARACTER SET = utf8mb3;

```

SHOW WARNINGS;

```

-----
-- Table `item_pedido`
-----

```

```

CREATE TABLE IF NOT EXISTS `item_pedido` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `ativado_item_pedido` TINYINT NOT NULL DEFAULT '0',
  `especificacoes_item_pedido` VARCHAR(100) NOT NULL,
  `produto_id` INT NOT NULL,

```

```

`pedido_cliente_id` INT NOT NULL,
PRIMARY KEY (`id`),
CONSTRAINT `fk_item_pedido_pedido_cliente1`
  FOREIGN KEY (`pedido_cliente_id`)
  REFERENCES `pedido_cliente` (`id`),
CONSTRAINT `fk_item_pedido_produto1`
  FOREIGN KEY (`produto_id`)
  REFERENCES `produto` (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 71
DEFAULT CHARACTER SET = utf8mb3;

```

```
SHOW WARNINGS;
```

```

-----
-- Table `pedido_fornecedor`
-----

```

```

CREATE TABLE IF NOT EXISTS `pedido_fornecedor` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `d_entrega_pedido_fornecedor` DATE NOT NULL,
  `d_realizado_pedido_fornecedor` DATE NOT NULL,
  `quantidade_pedido_fornecedor` INT NOT NULL,
  `item_pedido_pedido_fornecedor` VARCHAR(50) NOT NULL,
  `fornecedor_id` INT NOT NULL,
  `funcionario_id` INT NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_pedido_fornecedor_fornecedor1`
    FOREIGN KEY (`fornecedor_id`)
    REFERENCES `fornecedor` (`id`),
  CONSTRAINT `fk_pedido_fornecedor_funcionario1`
    FOREIGN KEY (`funcionario_id`)
    REFERENCES `funcionario` (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 36
DEFAULT CHARACTER SET = utf8mb3;

```

```
SHOW WARNINGS;
```

```

ALTER TABLE pedido_fornecedor DROP COLUMN d_realizado_pedido_fornecedor;
ALTER TABLE pedido_fornecedor ADD d_realizado_pedido_fornecedor TIMESTAMP
DEFAULT CURRENT_TIMESTAMP NOT NULL;

```

```

-----
-- Table `materia_prima`
-----

```

```

CREATE TABLE IF NOT EXISTS `materia_prima` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome_materia_prima` VARCHAR(50) NOT NULL,
  `d_validade_materia_prima` DATE NOT NULL,

```

```

`estoque_materia_prima` INT NOT NULL,
`min_estoque_materia_prima` INT NOT NULL,
`d_ult_reposicao_materia_prima` DATE NOT NULL,
`descricao_materia_prima` VARCHAR(100) NOT NULL,
`barracao_rua_materia_prima` VARCHAR(2) NOT NULL,
`barracao_andar_materia_prima` VARCHAR(2) NOT NULL,
`barracao_num_materia_prima` VARCHAR(2) NOT NULL,
`ativado_materia_prima` TINYINT NOT NULL DEFAULT '0',
`pedido_fornecedor_id` INT NOT NULL,
PRIMARY KEY (`id`),
CONSTRAINT `fk_materia_prima_pedido_fornecedor1`
  FOREIGN KEY (`pedido_fornecedor_id`)
  REFERENCES `pedido_fornecedor` (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 8
DEFAULT CHARACTER SET = utf8mb3;

```

SHOW WARNINGS;

```

-----
-- Table `oque_vai`
-----
CREATE TABLE IF NOT EXISTS `oque_vai` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `materia_prima_id` INT NOT NULL,
  `produto_id` INT NOT NULL,
  `quant_materia_prima` VARCHAR(10) NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_oque_vai_materia_prima1`
    FOREIGN KEY (`materia_prima_id`)
    REFERENCES `materia_prima` (`id`),
  CONSTRAINT `fk_oque_vai_produto1`
    FOREIGN KEY (`produto_id`)
    REFERENCES `produto` (`id`))
ENGINE = InnoDB
AUTO_INCREMENT = 323
DEFAULT CHARACTER SET = utf8mb3;

```

SHOW WARNINGS;

```

CREATE TABLE pedido_fornecedor_backup (
  -- Mantemos o ID como Chave Primária para indexação, mas SEM auto_increment
  id INT PRIMARY KEY,
  d_entrega_pedido_fornecedor DATE,
  quantidade_pedido_fornecedor INT,
  item_pedido_pedido_fornecedor VARCHAR(50),
  fornecedor_id INT,
  funcionario_id INT,

```

```

d_realizado_pedido_fornecedor TIMESTAMP,

-- COLUNA DE CONTROLE DO DBA: Registra o momento exato do arquivamento
data_arquivamento TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

SHOW WARNINGS;

CREATE TABLE entrega_fornecedor_backup (
-- Mantém o ID original como PK para garantir a busca rápida, mas sem gerar novos
números
id INT PRIMARY KEY,
valor_total_entrega_fornecedor DECIMAL(10,2), -- (Mantido o erro de digitação 'entrega'
caso esteja assim no seu banco)
valor_unt_entrega_fornecedor DECIMAL(10,2),
item_entrega_fornecedor VARCHAR(50),
quantidade_entrega_fornecedor INT, -- (Mantido o 'quantidade' para espelhar
sua tabela)
observacao_entrega_fornecedor VARCHAR(100),
fornecedor_id INT,
status_entrega_fornecedor TINYINT,

-- COLUNA DE CONTROLE DO DBA: Para você saber exatamente quando esse registro
virou arquivo
data_arquivamento TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

SHOW WARNINGS;

CREATE TABLE pedido_cliente_backup (
-- Mantém o ID original como Chave Primária para consultas rápidas de auditoria, mas
sem auto_increment
id INT PRIMARY KEY,
quantidade_pedido_cliente INT,
valor_unt_pedido_cliente DECIMAL(10,2),
valor_total_pedido_cliente DECIMAL(10,2),
prazo_final_pedido_cliente DATE,
op_pedido_cliente TINYINT,
funcionario_id INT,
cliente_id INT,
d_realizacao_pedido_cliente TIMESTAMP,

-- COLUNA DE CONTROLE DO DBA: Registra o momento exato em que o pedido virou
arquivo
data_arquivamento TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

SHOW WARNINGS;

```



```

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

select * from pedido_fornecedor;

ALTER TABLE pedido_cliente DROP COLUMN d_realizacao_pedido_cliente;
ALTER TABLE pedido_cliente ADD COLUMN d_realizacao_pedido_cliente TIMESTAMP
DEFAULT CURRENT_TIMESTAMP NOT NULL;

ALTER TABLE historico_vendas DROP COLUMN d_realizacao_historico_vendas;
ALTER TABLE historico_vendas ADD COLUMN d_realizacao_historico_vendas
TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL;

ALTER TABLE historico_vendas DROP COLUMN d_termينو_historico_vendas;
ALTER TABLE historico_vendas ADD COLUMN d_termינו_historico_vendas TIMESTAMP
DEFAULT CURRENT_TIMESTAMP NOT NULL;

ALTER TABLE historico_compra DROP COLUMN d_realizacao_historico_compra;
ALTER TABLE historico_compra ADD COLUMN d_realizacao_historico_compra
TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL;

ALTER TABLE entrega_fornecedor ADD COLUMN status_entrega_fornecedor TINYINT
NOT NULL DEFAULT 0;

alter table estoque_final add column status_estoque_final TINYINT NOT NULL DEFAULT 0;

ALTER TABLE produto ADD COLUMN foto_nome VARCHAR(225);
ALTER TABLE produto ADD COLUMN foto_localizacao VARCHAR(225);
ALTER TABLE produto ADD COLUMN foto_blob LONGBLOB;
ALTER TABLE produto drop column imagem_produto;

ALTER TABLE funcionario ADD COLUMN foto_nome VARCHAR(225);
ALTER TABLE funcionario ADD COLUMN foto_localizacao VARCHAR(225);
ALTER TABLE funcionario ADD COLUMN foto_blob LONGBLOB;
ALTER TABLE funcionario drop column foto_funcionario;

-- AUTOMAÇÕES

-- Essa trigger é acionada quando o pedido_cliente se torna uma ordem de produção, assim
criando um registro na tabela producao para que o lote seja rastreavel --

DELIMITER $$

CREATE TRIGGER trg_inicia_producao
AFTER UPDATE ON pedido_cliente
FOR EACH ROW

```

```

BEGIN
    -- Verifica se o campo 'status' mudou
    IF OLD.op_pedido_cliente <> NEW.op_pedido_cliente AND NEW.op_pedido_cliente = 1
    THEN
        INSERT INTO producao (
            etapa_producao,
            status_producao,
            acontecimento_producao,
            funcionario_id,    -- Campo da tabela 'producao'
            cliente_id,        -- Campo da tabela 'producao'
            pedido_cliente_id  -- Campo da tabela 'producao'
        ) VALUES (
            'Em aguardo...',
            '0',
            now(),
            NEW.funcionario_id, -- IMPORTADO do pedido alterado
            NEW.cliente_id,     -- IMPORTADO do pedido alterado
            NEW.id              -- IMPORTADO do ID do pedido alterado
        );
    END IF;
END$$

```

DELIMITER ;

-- Essa trigger é acionada quando o sistema finaliza o processo de verificação de entrada de matéria-prima, criando um registro no histórico de compras de como o pedido foi recebido e que ele realmente ocorreu --

DELIMITER \$\$

```

CREATE TRIGGER trg_regi
AFTER UPDATE ON entrega_fornecedor
FOR EACH ROW
BEGIN
    DECLARE v_d_entrega DATE;
    DECLARE v_d_realizacao TIMESTAMP; -- Mudado para TIMESTAMP para bater com o
ALTER TABLE

```

```

    -- Verifica se o status mudou para '1' (Concluído/Recebido)
    IF OLD.status_entrega_fornecedor <> NEW.status_entrega_fornecedor AND
    NEW.status_entrega_fornecedor = 1 THEN

```

```

        -- Bloco BEGIN/END com DECLARE CONTINUE HANDLER para evitar que o
        SELECT baleado trave o UPDATE

```

```

        BEGIN
            DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_d_entrega = NULL; --
aponta erros

```

```

        SELECT p.d_entrega_pedido_fornecedor, p.d_realizado_pedido_fornecedor
        INTO v_d_entrega, v_d_realizacao
        FROM pedido_fornecedor p
        WHERE p.item_pedido_pedido_fornecedor = NEW.item_entrega_fornecedor
        AND p.fornecedor_id = NEW.fornecedor_id
        LIMIT 1;
    END;

    -- Se não achou o pedido correspondente, usamos a data atual como garantia
    (fallback)
    IF v_d_realizacao IS NULL THEN
        SET v_d_realizacao = NOW();
    END IF;
    IF v_d_entrega IS NULL THEN
        SET v_d_entrega = CURDATE(); -- data atual do sistema
    END IF;

    -- Insere de fato no histórico
    INSERT INTO historico_compra (
        d_entrega_historico_compra,
        entrega_fornecedor_id,
        d_realizacao_historico_compra
    ) VALUES (
        v_d_entrega,
        NEW.id,
        v_d_realizacao
    );

    END IF;
END$$

DELIMITER ;

-- -----
-- Procedure

DELIMITER $$

CREATE PROCEDURE sp_arquivar_p_e_fornecedor(IN p_entrega_id int, p_pedido_id INT)
BEGIN
    -- Variável para verificar se o pedido realmente existe
    DECLARE v_existe INT DEFAULT 0;

    -- Verifica a existência do pedido antes de gastar processamento
    SELECT COUNT(*) INTO v_existe FROM pedido_fornecedor WHERE id = p_pedido_id;
    SELECT COUNT(*) INTO v_existe FROM entrega_fornecedor WHERE id =
p_entrega_id;

```

```

IF v_existe > 0 THEN

    -- Iniciamos a transação segura (Se algo falhar, o banco desfaz tudo)
    START TRANSACTION;

    -- -----
    -- teste

        INSERT INTO entrega_fornecedor_backup (id,
valor_total_entrega_fornecedor,valor_unt_entrega_fornecedor,item_entrega_fornecedor,qu
antidade_entrega_fornecedor,
observacao_entrega_fornecedor,fornecedor_id,status_entrega_fornecedor,data_arquivamen
to)
        SELECT id, valor_total_entrega_fornecedor, valor_unt_entrega_fornecedor,
item_entrega_fornecedor, quantidade_entrega_fornecedor,
        observacao_entrega_fornecedor, fornecedor_id, status_entrega_fornecedor, NOW()
        FROM entrega_fornecedor
        WHERE id = p_entrega_id;

        SAVEPOINT ponto_interessante;
        -- -----
        -- PASSO 1: Copiar os dados do pedido para a tabela de backup
        -- (Isso substitui ou complementa o que a trigger trg_backup_produto faria)
        INSERT INTO pedido_fornecedor_backup
(id,d_entrega_pedido_fornecedor,quantidade_pedido_fornecedor,item_pedido_pedido_forne
cedor,fornecedor_id,
funcionario_id,d_realizado_pedido_fornecedor,data_arquivamento)
        SELECT
id,d_entrega_pedido_fornecedor,quantidade_pedido_fornecedor,item_pedido_pedido_forne
cedor,fornecedor_id,
funcionario_id,d_realizado_pedido_fornecedor, NOW()
        FROM pedido_fornecedor
        WHERE id = p_pedido_id;

        -- PASSO 2: Deletar as vendas associadas a esse produto primeiro
        -- Obrigatório por conta da FOREIGN KEY na tabela vendas!
        DELETE FROM entrega_fornecedor WHERE id = p_entrega_id;

        -- PASSO 3: Deletar o produto da tabela operacional
        DELETE FROM pedido_fornecedor WHERE id = p_pedido_id;

        -- Sucesso total! Consolida as exclusões e o backup no disco
        COMMIT;

    END IF;
END$$

DELIMITER ;

```

-- Essa trigger serve para registrar uma venda no historico após ela ser classificada como concluída na tela de envios do dia

DELIMITER \$\$

```
CREATE TRIGGER trg_vendas
AFTER UPDATE ON pedido_cliente
FOR EACH ROW
BEGIN
```

-- 1. Declaramos uma variável local para armazenar o ID do estoque encontrado

```
DECLARE v_estoque_final_id INT UNSIGNED;
```

-- Vamos assumir que 'op_pedido_cliente' mudou para 2 (ex: Pedido Concluído/Entregue)

-- Vamos assumir que 'op_pedido_cliente' = 2 significa "Pedido Entregue/Venda Concluída"

```
IF OLD.op_pedido_cliente <> NEW.op_pedido_cliente AND NEW.op_pedido_cliente = 2
THEN
```

-- 2. Buscamos o ID do estoque final cruzando a produção vinculada a este pedido

```
SELECT e.id INTO v_estoque_final_id
FROM estoque_final e
INNER JOIN producao p ON e.producao_id = p.id
WHERE p.pedido_cliente_id = NEW.id
LIMIT 1;
```

```
INSERT INTO historico_vendas (
    d_entrega_historico_vendas,
    quantidade_historico_vendas,
    valor_total_historico_vendas,
    valor_unt_historico_vendas,
    prazo_final_historico_vendas,
    pedido_cliente_id,
    estoque_final_id, -- Nota: Garanta que esse ID exista ou mude a restrição da FK se
aceitar NULL
    d_realizacao_historico_vendas,
    d_termino_historico_vendas
) VALUES (
    CURDATE(), -- data de entrega (hoje)
    NEW.quantidade_pedido_cliente, -- quantidade
    NEW.valor_total_pedido_cliente, -- valor total
    NEW.valor_unt_pedido_cliente, -- valor unitário
    NEW.prazo_final_pedido_cliente, -- prazo final original
    NEW.id, -- ID do pedido
    v_estoque_final_id,
    NEW.d_realizacao_pedido_cliente, -- data de realização vinda do pedido
```

```
        NOW()                -- data de término (agora)
    );

    END IF;
END$$

DELIMITER ;
```